МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №5

Специальность ПО-13

Выполнил:
М.А.Шумило
студент группы
ПО-13
Проверил:
А.Д.Кулик
13.03.2026

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Ход работы

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Выполнение:

Код программы:

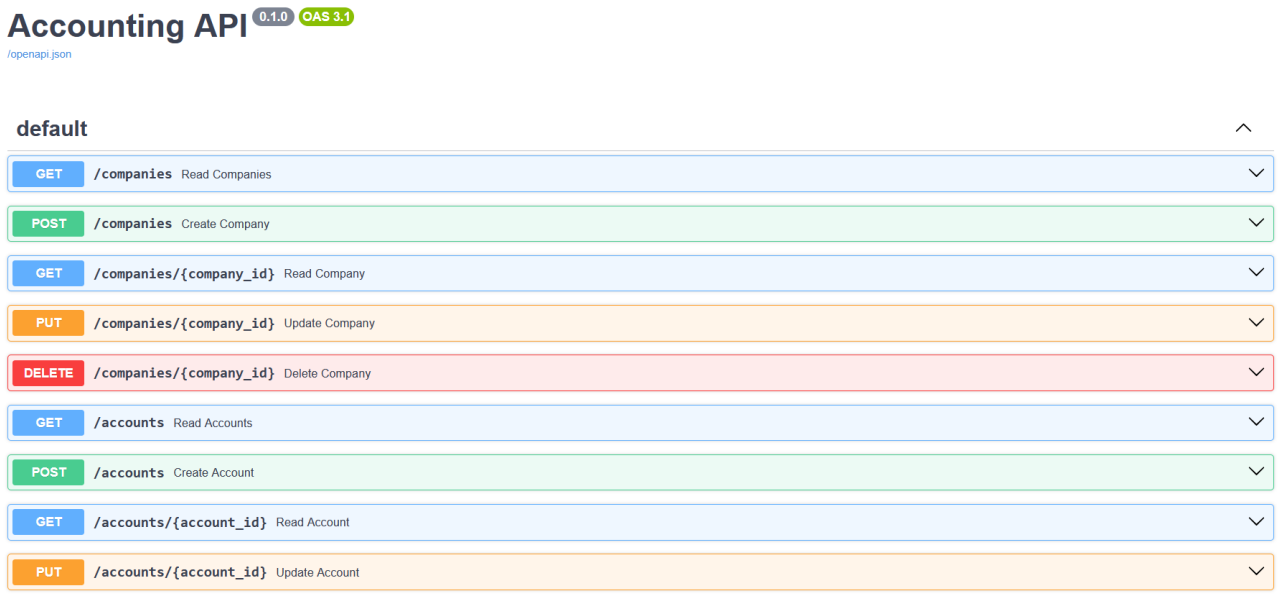
```
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session

from db import SessionLocal
from crud import companies, accounts, counterparties, documents, postings

from schemas.companies import CompanyCreate, CompanyUpdate, CompanyOut
from schemas.accounts import AccountCreate, AccountUpdate, AccountOut
from schemas.counterparties import CounterpartyCreate, CounterpartyUpdate,
CounterpartyOut
from schemas.documents import DocumentCreate, DocumentUpdate, DocumentOut
from schemas.postings import PostingCreate, PostingUpdate, PostingOut

app = FastAPI(title="Accounting API")

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# запуск uvicorn 1:app --reload
# COMPANIES

@app.get("/companies", response_model=list[CompanyOut])
def read_companies(db: Session = Depends(get_db)):
    return companies.get_all(db)

@app.get("/companies/{company_id}", response_model=CompanyOut)
def read_company(company_id: int, db: Session = Depends(get_db)):
    obj = companies.get(db, company_id)
    if not obj:
        raise HTTPException(404, "Company not found")
    return obj

@app.post("/companies", response_model=CompanyOut)
def create_company(data: CompanyCreate, db: Session = Depends(get_db)):
    return companies.create(db, data)

@app.put("/companies/{company_id}", response_model=CompanyOut)
```

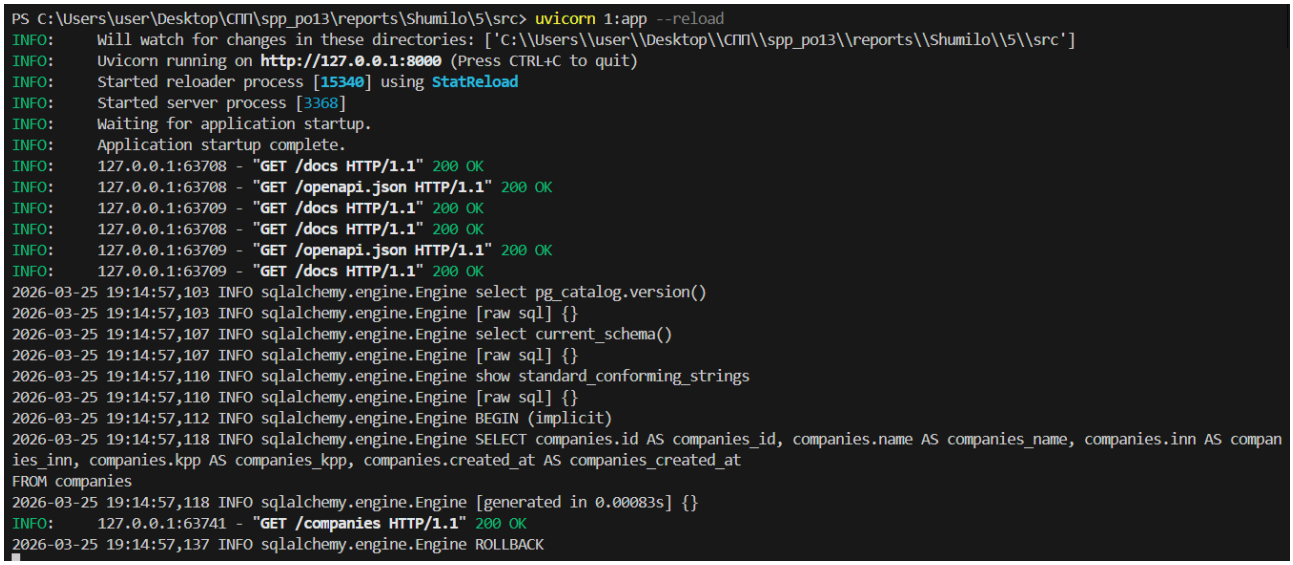
```
def update_company(company_id: int, data: CompanyUpdate, db: Session =
Depends(get_db)):
    obj = companies.update(db, company_id, data)
    if not obj:
        raise HTTPException(404, "Company not found")
    return obj

@app.delete("/companies/{company_id}")
def delete_company(company_id: int, db: Session = Depends(get_db)):
    ok = companies.delete(db, company_id)
    if not ok:
        raise HTTPException(404, "Company not found")
    return {"deleted": True}

# ACCOUNTS

@app.get("/accounts", response_model=list[AccountOut])
def read_accounts(db: Session = Depends(get_db)):
    return accounts.get_all(db)

@app.get("/accounts/{account_id}", response_model=AccountOut)
def read_account(account_id: int, db: Session = Depends(get_db)):
    obj = accounts.get(db, account_id)
    if not obj:
        raise HTTPException(404, "Account not found")
    return obj

@app.post("/accounts", response_model=AccountOut)
def create_account(data: AccountCreate, db: Session = Depends(get_db)):
    return accounts.create(db, data)

@app.put("/accounts/{account_id}", response_model=AccountOut)
def update_account(account_id: int, data: AccountUpdate, db: Session =
Depends(get_db)):
    obj = accounts.update(db, account_id, data)
    if not obj:
        raise HTTPException(404, "Account not found")
    return obj

@app.delete("/accounts/{account_id}")
def delete_account(account_id: int, db: Session = Depends(get_db)):
    ok = accounts.delete(db, account_id)
    if not ok:
        raise HTTPException(404, "Account not found")
    return {"deleted": True}

# COUNTERPARTIES

@app.get("/counterparties", response_model=list[CounterpartyOut])
def read_counterparties(db: Session = Depends(get_db)):
    return counterparties.get_all(db)

@app.get("/counterparties/{counterparty_id}", response_model=CounterpartyOut)
def read_counterparty(counterparty_id: int, db: Session = Depends(get_db)):
    obj = counterparties.get(db, counterparty_id)
    if not obj:
        raise HTTPException(404, "Counterparty not found")
    return obj

@app.post("/counterparties", response_model=CounterpartyOut)
def create_counterparty(data: CounterpartyCreate, db: Session =
Depends(get_db)):
    return counterparties.create(db, data)
```

```
@app.put("/counterparties/{counterparty_id}", response_model=CounterpartyOut)
def update_counterparty(counterparty_id: int, data: CounterpartyUpdate,
                        db: Session = Depends(get_db)):
    obj = counterparties.update(db, counterparty_id, data)
    if not obj:
        raise HTTPException(404, "Counterparty not found")
    return obj

@app.delete("/counterparties/{counterparty_id}")
def delete_counterparty(counterparty_id: int, db: Session = Depends(get_db)):
    ok = counterparties.delete(db, counterparty_id)
    if not ok:
        raise HTTPException(404, "Counterparty not found")
    return {"deleted": True}

# --- DOCUMENTS ---

@app.get("/documents", response_model=list[DocumentOut])
def read_documents(db: Session = Depends(get_db)):
    return documents.get_all(db)

@app.get("/documents/{document_id}", response_model=DocumentOut)
def read_document(document_id: int, db: Session = Depends(get_db)):
    obj = documents.get(db, document_id)
    if not obj:
        raise HTTPException(404, "Document not found")
    return obj

@app.post("/documents", response_model=DocumentOut)
def create_document(data: DocumentCreate, db: Session = Depends(get_db)):
    return documents.create(db, data)

@app.put("/documents/{document_id}", response_model=DocumentOut)
def update_document(document_id: int, data: DocumentUpdate, db: Session =
Depends(get_db)):
    obj = documents.update(db, document_id, data)
    if not obj:
        raise HTTPException(404, "Document not found")
    return obj

@app.delete("/documents/{document_id}")
def delete_document(document_id: int, db: Session = Depends(get_db)):
    ok = documents.delete(db, document_id)
    if not ok:
        raise HTTPException(404, "Document not found")
    return {"deleted": True}

# --- POSTINGS ---

@app.get("/postings", response_model=list[PostingOut])
def read_postings(db: Session = Depends(get_db)):
    return postings.get_all(db)

@app.get("/postings/{posting_id}", response_model=PostingOut)
def read_posting(posting_id: int, db: Session = Depends(get_db)):
    obj = postings.get(db, posting_id)
    if not obj:
        raise HTTPException(404, "Posting not found")
    return obj

@app.post("/postings", response_model=PostingOut)
def create_posting(data: PostingCreate, db: Session = Depends(get_db)):
    return postings.create(db, data)
```

```
@app.put("/postings/{posting_id}", response_model=PostingOut)
def update_posting(posting_id: int, data: PostingUpdate, db: Session =
Depends(get_db)):
    obj = postings.update(db, posting_id, data)
    if not obj:
        raise HTTPException(404, "Posting not found")
    return obj

@app.delete("/postings/{posting_id}")
def delete_posting(posting_id: int, db: Session = Depends(get_db)):
    ok = postings.delete(db, posting_id)
    if not ok:
        raise HTTPException(404, "Posting not found")
    return {"deleted": True}
```

Рисунки с результатами работы программы





```
PS C:\Users\user\Desktop\СПП\spp_po13\reports\Shumilo\5\src> uvicorn 1:app --reload
INFO:     Will watch for changes in these directories: ['C:\\Users\\user\\Desktop\\СПП\\spp_po13\\reports\\Shumilo\\5\\src']
INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:     Started reloader process [15340] using StatReload
INFO:     Started server process [3368]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     127.0.0.1:63708 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:63708 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:63709 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:63708 - "GET /docs HTTP/1.1" 200 OK
INFO:     127.0.0.1:63709 - "GET /openapi.json HTTP/1.1" 200 OK
INFO:     127.0.0.1:63709 - "GET /docs HTTP/1.1" 200 OK
2026-03-25 19:14:57,103 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2026-03-25 19:14:57,103 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-03-25 19:14:57,107 INFO sqlalchemy.engine.Engine select current_schema()
2026-03-25 19:14:57,107 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-03-25 19:14:57,110 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2026-03-25 19:14:57,110 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-03-25 19:14:57,112 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-03-25 19:14:57,118 INFO sqlalchemy.engine.Engine SELECT companies.id AS companies_id, companies.name AS companies_name, companies.inn AS compan
ies_inn, companies.kpp AS companies_kpp, companies.created_at AS companies_created_at
FROM companies
2026-03-25 19:14:57,118 INFO sqlalchemy.engine.Engine [generated in 0.00083s] {}
INFO:     127.0.0.1:63741 - "GET /companies HTTP/1.1" 200 OK
2026-03-25 19:14:57,137 INFO sqlalchemy.engine.Engine ROLLBACK
```

Вывод: приобрёл практические навыки разработки API и баз данных